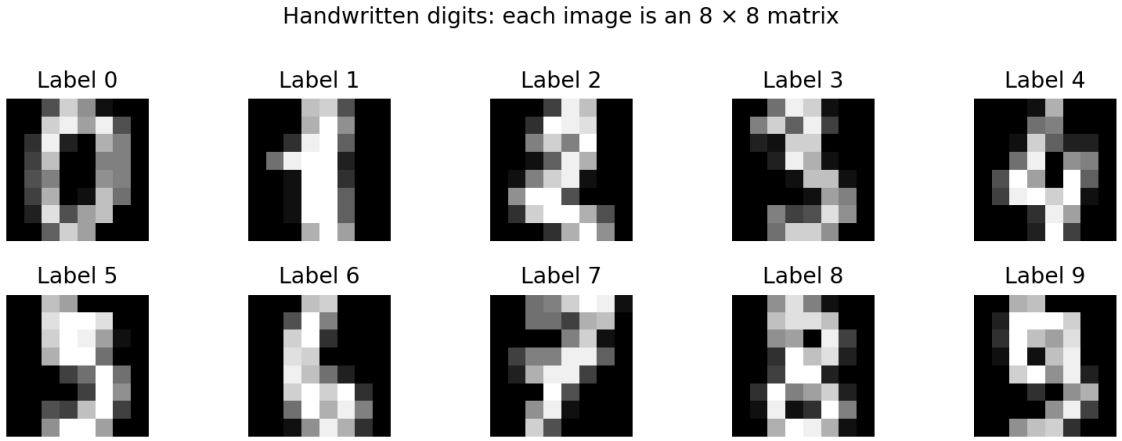
פרק 13 — רשתות קונבולוציה (CNN)

מפיקסלים ומסננים לרשת עצבית שמזהה דפוסים בתמונה

ספר הקורס המאוחד | מסלול התמחות למידת מכונה ולמידה עמוקה, 5 יחידות | היקף פרק בתכנית: 50 שעות | מהדורת תלמיד



איור: דוגמאות מתמונות ספרות. כל תמונה היא מערך של 64 עוצמות אפור.

לפני שמתחילים — מטרות, ידע קודם ומעמד התוכן

בסיום הפרק תוכלו להסביר מדוע רשת צפופה אינה מנצלת את המבנה המרחבי של תמונה באופן מובנה; לחשב מסנן על חלון תמונה; לחזות את ממדי התוצר של שכבות Convolution ו־Pooling; לתאר כיצד מסננים נלמדים באימון; ולבנות רשת CNN קטנה, להעריך אותה ולנתח טעויות.

הפרק ממשיך את פרק 6 (עיבוד תמונה), פרק 11 (נגזרות וגרדיאנטים) ופרק 12 (רשתות עצביות). מומלץ לחזור על מטריצה דו־ממדית, כפל וחיבור, מכפלה פנימית, ReLU, Cross-Entropy ונתוני Train/Validation/Test. לא נניח שידיעת Python לבדה מספיקה להבנת הרשת.

ידע קודם	איפה השתמשנו בו	איך ישמש בפרק
מטריצות ו־RGB	פרק 6	חלונות, מסננים וערוצים
מכפלה פנימית ומשקולות	פרקים 1 ו־9	כל תא במפת מאפיינים
כלל השרשרת ו־Gradient Descent	פרק 11	למידת משקולות המסננים
Forward / Backward	פרק 12	אימון רשת מקצה לקצה
הערכה וולידציה	פרקים 5.1 ו־5.2	בדיקה הוגנת של CNN

עוגן רשמי לפי מפת המאגר: התמחות למידת מכונה ולמידה עמוקה, פרק 13 — *CNN, 50* שעות. כיסוי תתי־הנושאים מוצע בהלימה מקצועית; יש להצליב עם נוסח משרד החינוך התקף. חומרים מתקדמים יסומנו כהעמקה.

1. סיפור פתיחה — כיצד טלפון מזהה ספרה בכתב יד?

דודו מפתח יישומון לסריקת מספרי חדרים בבית הספר. לפעמים התמונה מוארת, לפעמים היא מעט מוזזת, ולפעמים הספרה נכתבת בקו עבה. אם נצמיד לכל פיקסל משקל שונה ברשת צפופה, הרשת

תצטרך ללמוד בנפרד שמאפיין מסוים יכול להופיע גם למעלה וגם למטה. אבל קצה, קו או פינה הם אותם דפוסים חזותיים גם כשהם זזים מעט.

מכאן עולה רעיון מרכזי: במקום לתת לכל מיקום בתמונה מערכת משקולות נפרדת, נסרוק את התמונה בעזרת מסנן קטן שמשקולותיו משותפות לכל המיקומים. כך נלמד לזהות דפוס מקומי, נשמור על קשרים בין פיקסלים שכנים ונקטין לעיתים קרובות את מספר הפרמטרים. זו אחת מאבני היסוד של CNN.

לבדיקה עצמית: אם אותו קו אנכי נמצא פעם בצד שמאל ופעם בצד ימין, מדוע כדאי להשתמש באותו מסנן בשני המקומות?

2. לפני הרשת: תמונה היא טנזור ולא ציור מסתורי

תמונת גווני אפור אפשר לראות כמטריצה בגובה H וברוחב W. ערך בכל תא מתאר עוצמת אור. בתמונת RGB יש שלוש מטריצות — אחת לכל ערוץ צבע. במהלך אימון הרשתות אנו עובדים בדרך כלל עם אוסף תמונות (Batch), ולכן למערך הנתונים יש ארבעה ממדים.

יש להבחין בין שמות הממדים לבין סדרם בספרייה. ב־PyTorch סדר ברירת המחדל הוא N,C,H,W: מספר תמונות, מספר ערוצים, גובה ורוחב. במערכות אחרות תיתכן צורה N,H,W,C. בלבול בסדר הערוצים גורם לאחת משגיאות הקוד השכיחות בתחום.

Grayscale: (H, W) | RGB: (H, W, 3) | PyTorch batch: (N, C, H, W)

למשל, אצווה של 32 תמונות צבע בגודל 64×64 תוצג ב־PyTorch בממדים (32,3,64,64). מספר הערכים הגולמיים הכולל הוא 32×3×64×64. רשת CNN לומדת להפוך את הפיקסלים למפות מאפיינים, ובהמשך להחלטה על מחלקת התמונה.

```
import numpy as np
from sklearn.datasets import load_digits

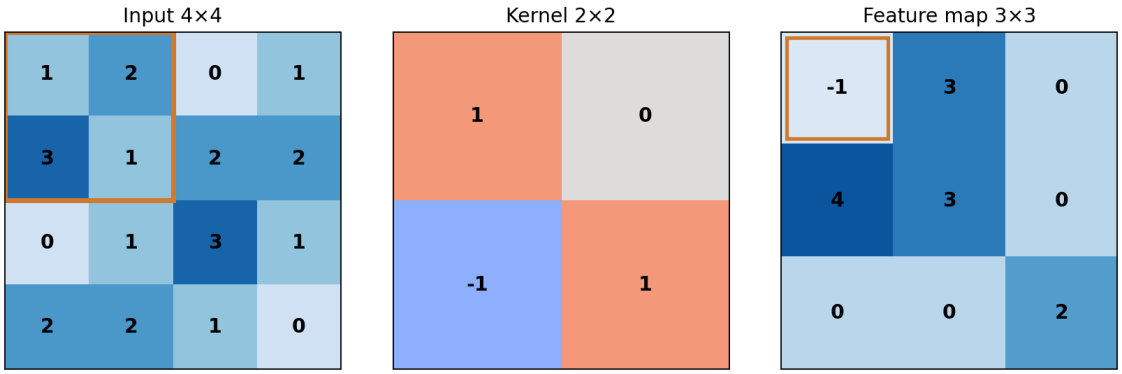
digits = load_digits()
image = digits.images[0]
print(image.shape)         # (8, 8)
print(image.min(), image.max())  # intensities in [0, 16]
print(image[0, :])         # first row of pixels
```

לבדיקה עצמית: מה ההבדל בין מספר הפיקסלים בתמונה לבין מספר המאפיינים המופקים בשכבת CNN?

3. מהו מסנן (Kernel) ומה הוא עושה?

מסנן הוא מטריצה קטנה של משקולות, לדוגמה 2×2 או 3×3. מניחים אותו מעל אזור מקומי בתמונה, מכפילים כל ערך תמונה במשקל המתאים במסנן ומחברים. התוצאה היא מספר יחיד: תגובת המסנן באזור הזה. לאחר מכן מזיזים את המסנן למיקום הבא וחוזרים על החישוב. כל התוצאות יחד יוצרות מפת מאפיינים (Feature Map).

במדעי התמונה נעשה שימוש במסננים קבועים, למשל להדגשת קצוות. ברשת CNN משקולות המסנן יכולות להשתנות בזמן האימון. לכן ההבדל החשוב אינו רק הפעולה של החלקת חלון, אלא העובדה שהמערכת לומדת איזה מסנן מועיל למשימה.



איור: חלון 2×2 עובר על מטריצה 4×4 ומפיק מפת מאפיינים 3×3.

4. חישוב ידני מלא — ערך ראשון במפת המאפיינים

ניקח חלון תמונה בגודל 2×2 והמסנן הבא. בכל תא במכפלה יש כפל אחד, ולאחריו מחברים את ארבע התוצאות. שימו לב לסימן השלילי במשקל השמאלי־תחתון של המסנן: הוא יכול להפחית מן התגובה כאשר הערך באזור הזה גבוה.

Window = [[1, 2], [3, 1]] Kernel = [[1, 0], [-1, 1]]

Output[0,0] = 1×1 + 2×0 + 3×(-1) + 1×1 = -1

כאשר המסנן זז פיקסל אחד ימינה, החלון הבא הוא [[2,0],[1,2]]. כעת מחשבים 2×0+1×1+0×(-1)+1×2=3. יש להשלים ידנית את יתר המיקומים כדי להבין מדוע גודל מפת התוצאה הוא 3×3. זו דוגמה של קורלציה צולבת (Cross-Correlation): המסנן אינו מתהפך לפני החישוב. בספריות CNN מקובל לקרוא לפעולה Convolution, אף שבמתמטיקה הקונבולוציה הפורמלית מבצעת היפוך של המסנן.

Y = [[-1, 3, 0], [4, 3, 0], [0, 0, 2]]

```
import numpy as np

image = np.array([[1, 2, 0, 1],
                  [3, 1, 2, 2],
                  [0, 1, 3, 1],
                  [2, 2, 1, 0]])
kernel = np.array([[1, 0], [-1, 1]])

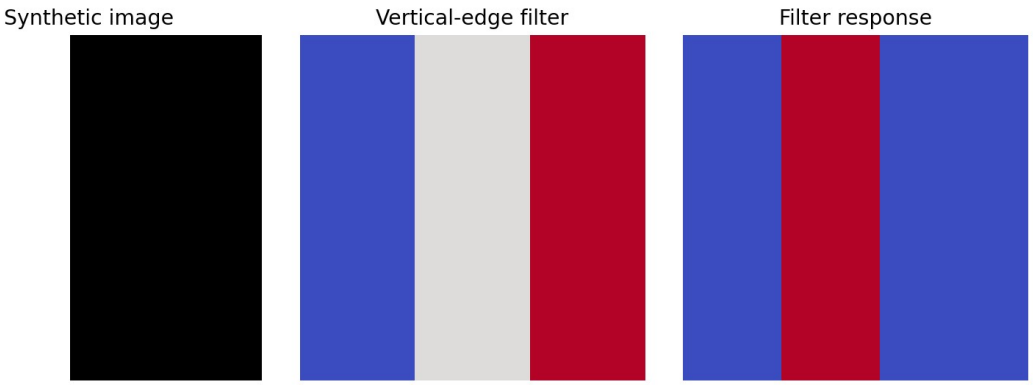
out = np.zeros((3, 3), dtype=int)
for row in range(3):
    for col in range(3):
        window = image[row:row+2, col:col+2]
        out[row, col] = np.sum(window * kernel)
print(out)
```

דיוק מתמטי: הקוד והדוגמה מבצעים *Cross-Correlation* ולא קונבולוציה עם היפוך מסנן. זו אותה מוסכמה שבה משתמשת *nn.Conv2d* ב*PyTorch*.

5. איך מסנן מזהה קצה או קו?

נניח שאזור שמאלי בתמונה כהה ואזור ימני בהיר. שינוי חד בין האזורים עשוי להצביע על קצה של עצם. מסנן שקובע משקלים שליליים בצד שמאל וחיוביים בצד ימין מגיב למעבר כזה. אין למסנן הבנה של המושג ״דלת״ או ״ספרה״; הוא רק מחבר הפרשים מקומיים על פי משקולות.

מסנן יחיד עשוי להגיב לקצה אנכי. מסננים אחרים עשויים להגיב לקווים אופקיים או לטקסטורות. בשכבות עמוקות יותר הרשת יכולה לשלב תגובות מקומיות ולהבחין בדפוסים מורכבים. התיאור הוא אינטואיטיבי: בפועל לא כל מסנן לומד תכונה שקל לתת לה שם.



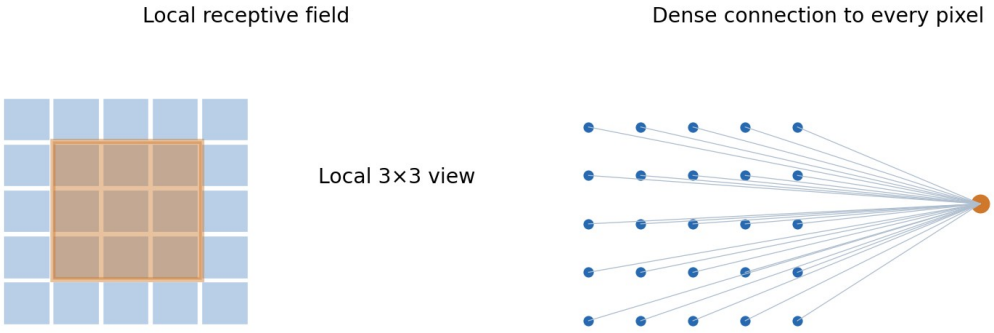
איור: דימוי של מסנן המדגיש מעבר בין אזור כהה ובהיר.

לבדיקה עצמית: אם נכפיל את כל ערכי התמונה ב־2, מה יקרה לתגובת המסנן הזה, בהנחה שאין **Bias** ואין **Clipping**?

6. מדוע שיתוף משקולות כל כך חשוב?

ברשת צפופה (Dense) כל נוירון יכול להיות מחובר לכל ערכי הקלט, ולכן ניתן ללמוד משקל שונה לכל מיקום. ב־CNN המסנן הקטן מחובר רק לסביבה מקומית, אך משתמש באותן משקולות כשהוא עובר על מיקומים שונים. השילוב בין קישוריות מקומית לשיתוף משקולות מאפשר לנצל את המבנה המרחבי.

לדוגמה, אם תמונה מכילה 64×64×3 ערכים ולנוירון צפוף יחיד יש משקל לכל ערך, דרושים 12,288 משקולות ועוד הטיה. לעומת זאת מסנן 3×3 המקבל שלושה ערוצי קלט דורש 3×3×3=27 משקולות ועוד הטיה; הוא מוחל שוב ושוב לאורך התמונה. ההשוואה היא בין נוירון צפוף יחיד למסנן יחיד, ולא בין שתי ארכיטקטורות שלמות.



איור: קישוריות מקומית לעומת קישוריות מלאה למאפייני הקלט.

One 3×3 RGB filter: 3 × 3 × 3 + 1 bias = 28 parameters

7. ממדי הפלט: Kernel Size, Stride, Padding

אחת המיומנויות החשובות לבגרות ולפרויקט היא היכולת לחזות את ממדי המערך לאחר שכבת עיבוד. יש שלושה גדלים מרכזיים: גודל המסנן K, גודל הצעד S (Stride) ומספר ערכי הריפוד P (Padding) שמוסיפים בכל צד.

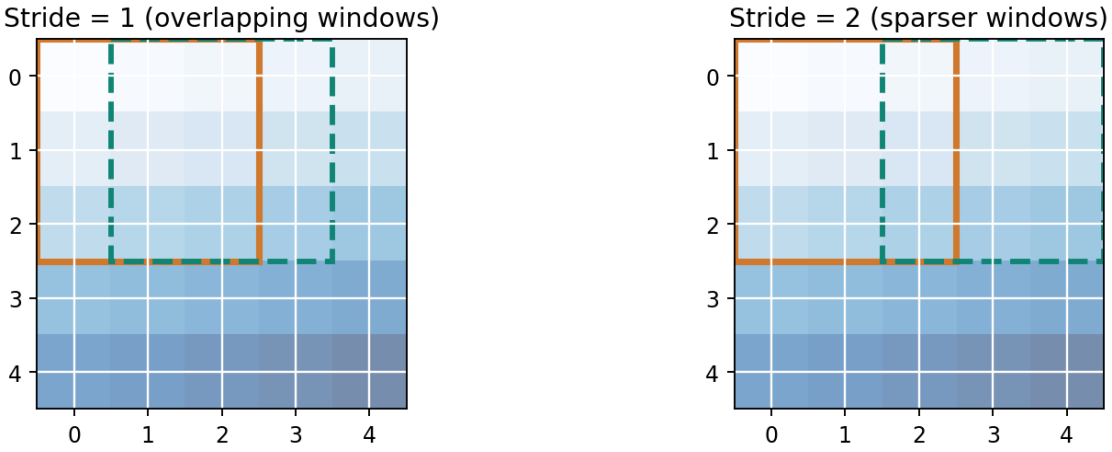
כאשר המסנן נע בצעדים גדולים יותר, מספר המקומות שבהם הוא נסרק קטן. כאשר מוסיפים ריפוד אפסים מסביב לתמונה, אפשר לחשב תגובות סמוך לשוליים ולשמר טוב יותר את הממדים. חשוב לזכור: Padding אינו משחזר מידע שלא היה בתמונה; הוא מוסיף ערכים לפי כלל שנקבע.

$$\text{H_out} = \text{floor}((H + 2P - K) / S) + 1 \quad [\text{dilation} = 1]$$

$$\text{W_out} = \text{floor}((W + 2P - K) / S) + 1 \quad [\text{dilation} = 1]$$

דוגמה 1: תמונה 8×8, מסנן 3×3, צעד 1, בלי ריפוד: H_out=(8−3)/1+1=6 ולכן הפלט 6×6. דוגמה 2: אותה תמונה עם ריפוד 1 בכל צד וצעד 1: H_out=(8+2−3)+1=8. דוגמה 3: תמונה 8×8, מסנן 3×3, ריפוד 1, צעד 2: 7214 . בכל שלוש הדוגמאות מספר ערוצי הפלט נקבע לפי מספר המסננים, לא לפי מספר ערוצי הקלט.

קלט H×W	K	Stride	Padding בכל צד	פלט H×W
8×8	3	1	0	6×6
8×8	3	1	1	8×8
8×8	3	2	1	4×4
28×28	5	1	0	24×24



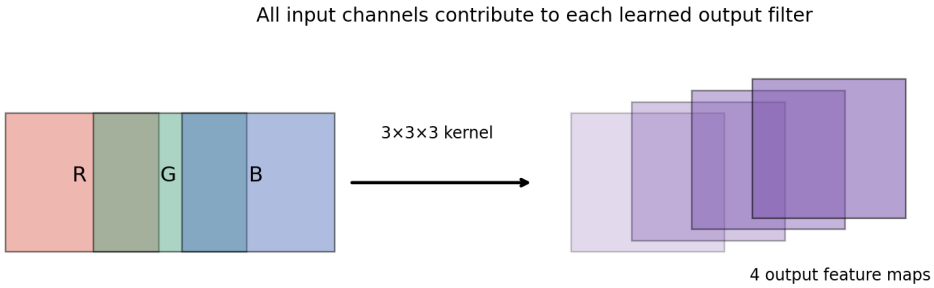
איור: *Stride* גדול יותר מדלג על מיקומי סריקה ומקטין את ממדי הפלט.

לבדיקה עצמית: חישבו את גודל הפלט של תמונה 32×32 עם מסנן 5×5, ריפוד 2 וצעד 2. נמקו את פעולת הרצפה.

8. לאן נעלמים ערוצי הצבע? Convolution רב־ערוצית

תמונת RGB מכילה שלושה ערוצים. מסנן CNN אחד אינו מסנן של ערוץ בודד בלבד: בדרך כלל הוא מכיל קבוצת משקולות לכל ערוץ קלט. מחשבים תגובה מכל ערוץ, מחברים את שלושת התרומות ומוסיפים Bias; התוצאה היא מפת מאפיינים אחת.

אם נבחר שמונה מסננים שונים, נקבל שמונה מפות מאפיינים. כל מסנן יכול ללמוד דפוס אחר. אם לקלט יש 3 ערוצים ולשכבה 8 מסננים בגודל 3×3, מספר הפרמטרים הוא 8×(3×3×3+1)=224. ממדי הפלט יהיו N,8,H_out,W_out.



איור: כל מסנן פועל על כל ערוצי הקלט ויוצר ערוץ פלט אחד.

Conv parameters = C_out × (C_in × K_h × K_w + 1 bias)

לצורך ההבנה, חשבו ידנית פיקסל פלט של מסנן 1×1 כאשר תרומת שלושת הערוצים היא R×0.2+G×0.5+B×(0.1)+0.3. אין משמעות הדבר שהרשת מפרשת כל ערוץ כמשתנה עצמאי ללא הקשר; המשקולות נלמדות יחד בתהליך האימון.

9. פונקציית הפעלה — מדוע Convolution לבדה אינה מספיקה?

שכבת קונבולוציה מחשבת צירוף ליניארי מקומי של הערכים. אם נחבר רק שכבות ליניאריות, ללא פעולות לא־ליניאריות ביניהן, המערכת עדיין תהיה צירוף ליניארי כולל (בהתעלם מפעולות כמו Pooling). לכן מוסיפים לרוב פונקציית הפעלה כגון ReLU: ערכים שליליים הופכים לאפס וערכים חיוביים נשארים כפי שהם.

ReLU מאפשרת לשלב דפוסים בצורה לא־ליניארית. בגרף, החצי השמאלי נשאר באפס והחצי הימני עולה בקו ישר. אולם ReLU לבדה אינה פותרת כל בעיה, ויש לדון גם במקרים שבהם נוירונים נשארים לא פעילים לאורך האימון.

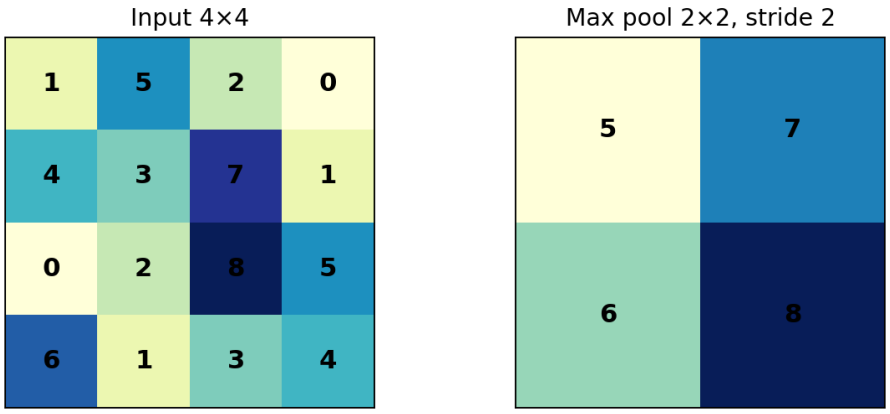
ReLU(z) = max(0, z)

ReLU(-2)=0 ReLU(0)=0 ReLU(3.5)=3.5

10. Pooling — כיצד מצמצמים מפה בלי לאבד כל דפוס מקומי?

Pooling מסכמת אזור קטן במפת מאפיינים לערך יחיד. ב־Max Pooling לוקחים את הערך המרבי בכל חלון. פעולה זו מקטינה את הרזולוציה ויכולה להקטין רגישות לשינויים מקומיים קטנים, אך עלולה גם למחוק מידע מרחבי מדויק. אין להתייחס אליה כאל פעולה שמשפרת כל מודל בכל מצב.

נחשב לדוגמה Max Pooling בגודל 2×2 עם Stride 2: נפרק מטריצה 4×4 לארבעה בלוקים לא חופפים. מכל בלוק נבחר את הערך הגבוה ביותר. התוצאה היא מטריצה 2×2. אין כאן משקולות שהשכבה לומדת; כלל הבחירה מוגדר מראש.



איור: *Max Pooling* בוחר את הערך הגדול בכל חלון 2×2.

MaxPool([[1,5],[4,3]]) = 5

```
import torch
import torch.nn.functional as F

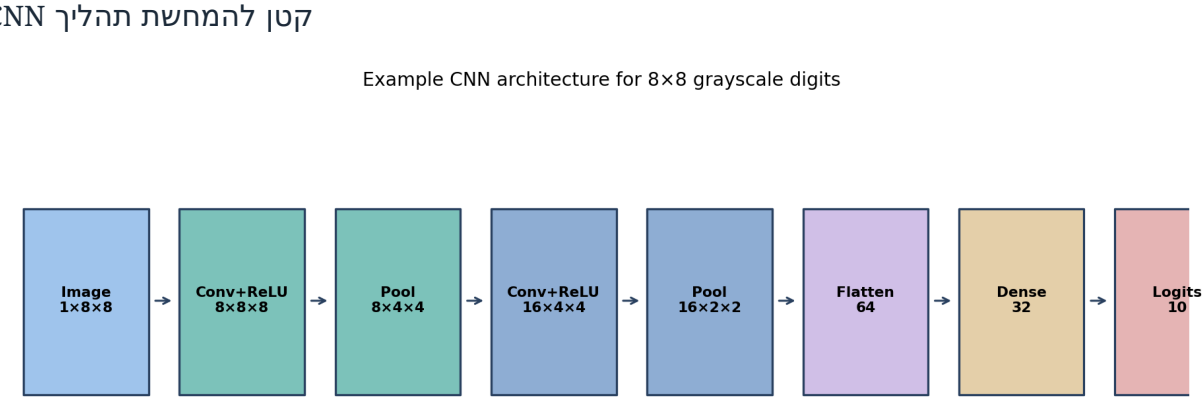
x = torch.tensor([[[[1., 5., 2., 0.],
                    [4., 3., 7., 1.],
                    [0., 2., 8., 5.],
                    [6., 1., 3., 4.]]]])
print(F.max_pool2d(x, kernel_size=2, stride=2))
```

לבדיקה עצמית: אם היינו משתמשים ב־**Average Pooling** במקום **Max Pooling**, איזה מידע היה מודגש יותר? חשבו את הבלוק הראשון.

11. מהי ארכיטקטורת CNN מלאה?

CNN היא לא שכבה אחת אלא שרשרת פעולות. בשלב הראשון הקלט הוא תמונה. שכבות Convolution לומדות דפוסים מקומיים, פונקציות ReLU מוסיפות אי־ליניאריות, ו־Pooling עשוי להקטין את המפות. בסוף ניתן לשטח את מפות המאפיינים לווקטור ולחבר שכבות Dense שמפיקות ערכי תחזית.

בדוגמה שנשתמש בה נבנה רשת קטנה לסיווג ספרות: 1×8×8 → קונבולוציה ל־8 מפות → Pooling ל־8×4×4 → קונבולוציה ל־16 מפות → Pooling ל־16×2×2 → Flatten באורך 64 → שכבה חבויה בגודל 32 → 10 ערכי פלט. המודל אינו נחשב בהכרח ״עמוק״ לפי כל סף מחקרי; זהו מבנה לימודי קטן להמחשת תהליך CNN.

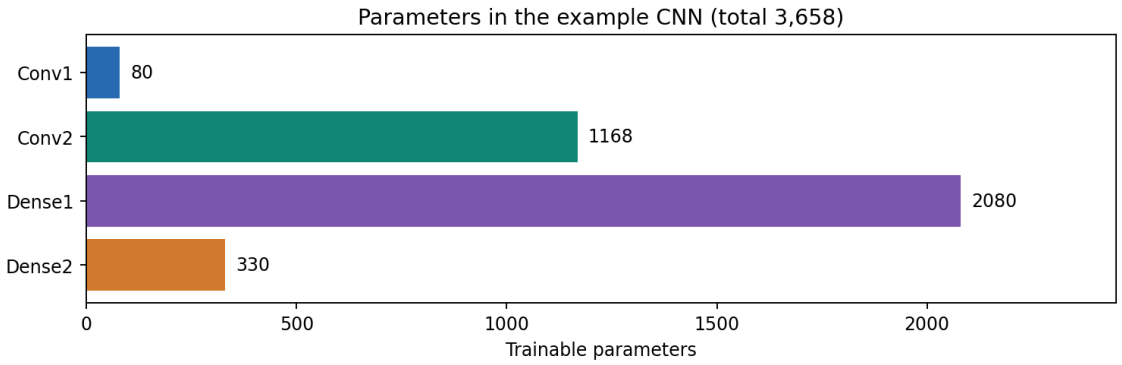


איור: ממדי המערכים ברשת *CNN* לימודית לספרות 8×8.

12. סופרים פרמטרים — יכולת שחזור והבנת הארכיטקטורה

ספירת פרמטרים מאפשרת לזהות טעויות בתכנון שכבות. בשכבת Conv2d המשקולות הן בגודל (מספר מסנני פלט, מספר ערוצי קלט, גובה המסנן, רוחב המסנן), ולכל מסנן יש בדרך כלל Bias אחד. בשכבת Dense מספר המשקולות הוא מספר הקלטים כפול מספר הנוירונים, ועוד Bias לכל נוירון.

שכבה	נוסחת חישוב	מספר פרמטרים
Conv1: 1→8, 3×3	(3+1×3×1)×8	80
Conv2: 816, 3×3	(3+1×3×8)×16	1,168
Dense1: 64→32	32+32×64	2,080
Dense2: 3210	10+10×32	330
סך הכל	80+1,168+2,080+330	3,658



איור: השוואת מספר הפרמטרים בארכיטקטורה הלימודית.

13. Forward Pass, Loss, Backpropagation — כיצד רשת לומדת?

במעבר קדימה (Forward Pass) התמונה עוברת דרך כל השכבות ונוצרים עשרה מספרים הנקראים Logits: מספר אחד לכל מחלקת ספרה. אלה אינם הסתברויות בפני עצמם. פונקציית Softmax יכולה להמיר אותם לערכים בין 0 ל־1 שסכומם 1, אך בזמן אימון מסווג רב־מחלקתי משתמשים לרוב ב־CrossEntropyLoss שמקבלת את ה־Logits ישירות.

לאחר השוואת ה־Logits לתווית האמיתית, פונקציית האובדן מספקת מספר המייצג עד כמה התחזית אינה מתאימה לתווית. באמצעות Backpropagation וכלל השרשרת מחשבים את נגזרת האובדן ביחס למשקל בכל שכבה. האופטימייזר משנה מעט את המשקולות בכיוון שנועד להקטין את השגיאה. זהו אותו עקרון מתמטי שתרגלנו בפרק 11, הפעם על מספר רב של מסננים ושכבות.

$$\text{Loss} = -\log\left(\frac{\exp(z_y)}{\sum_j \exp(z_j)}\right)$$

נוסחת Cross-Entropy לדוגמה אחת משתמשת ב־z_y שהוא ה־Logit של המחלקה האמיתית. אם למודל שלוש מחלקות וה־Logits הם [2,1,0], עבור תווית 0 ההסתברות אחרי Softmax היא בקירוב 0.6652 ולכן האובדן הוא בערך 0.4076. אם תווית האמת הייתה 2, האובדן היה גדול יותר. החישוב נועד לבנות אינטואיציה, לא לדרוש פיתוח של כל נגזרות Softmax בשלב זה.

$$P(y=0) = e^2 / (e^2+e^1+e^0) \approx 0.6652$$

$$\text{CrossEntropy}(y=0) = -\ln(0.6652) \approx 0.4076$$

```
import torch
```

```
import torch.nn as nn

logits = torch.tensor([[2.0, 1.0, 0.0]])
labels = torch.tensor([0])
loss = nn.CrossEntropyLoss()(logits, labels)
print(round(loss.item(), 4))  # approximately 0.4076
```

14. מימוש רשת CNN אמיתית באמצעות PyTorch

במימוש הבא נשתמש ב־PyTorch, משום שהקוד מאפשר לזהות באופן מפורש את שכבות הרשת ואת תהליך האימון. זהו קוד לימודי, לא מתכון יחיד לפרויקט. נשתמש במאגר `digits` הזמין ב־scikit-learn ללא הורדה חיצונית. תמונה היא 8×8 בגווני אפור, הערכים המקוריים בין 0 ל־16, והתווית היא אחת מעשר הספרות.

נחלק את הנתונים באופן מדורג ל־Train, Validation ו־Test, עם שמירה בקירוב על התפלגות המחלקות. את קבוצת ה־Validation נשתמש כדי לבחור למשל מספר Epochs ואת Test נשמור להערכה סופית. הנרמול לפי תחום מדידה ידוע [0,16] אינו מחשב סטטיסטיקה מכל המאגר, ולכן אין כאן התאמת Scaler על Test.

```
import numpy as np
import torch
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(7)
np.random.seed(7)
digits = load_digits()
X = digits.images.astype(np.float32) / 16.0
y = digits.target.astype(np.int64)

X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=7)
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, test_size=0.18, stratify=y_dev, random_state=7)

train_x = torch.tensor(X_train[:, None, :, :])
train_y = torch.tensor(y_train)
val_x = torch.tensor(X_val[:, None, :, :])
val_y = torch.tensor(y_val)
print(train_x.shape, val_x.shape)
```

השימוש בממד חדש `None` מוסיף ערוץ צבע יחיד, כדי לקבל את סדר הממדים המתאים ל־Conv2d: מספר הדוגמאות, מספר הערוצים, גובה ורוחב. אין לשטח תמונה לווקטור לפני שכבת הקונבולוציה, מפני שהמטרה היא לנצל את השכנות המרחבית.

```
import torch.nn as nn

class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 2 * 2, 32),
            nn.ReLU(),
            nn.Linear(32, 10)
```

```
        )

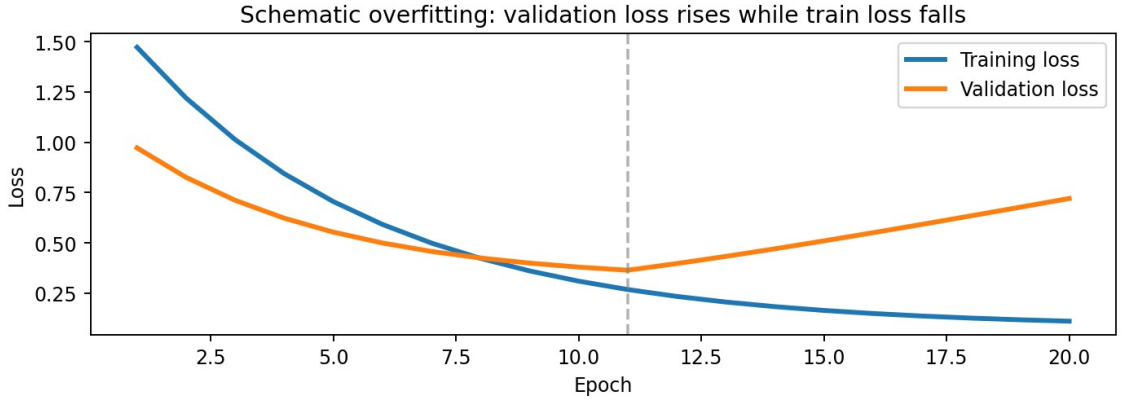
    def forward(self, x):
        return self.classifier(self.features(x))

model = DigitCNN()
print(model(torch.zeros(2, 1, 8, 8)).shape)  # (2, 10)
print(sum(p.numel() for p in model.parameters()))  # 3658
```

15. אימון מבוקר: מדוע צריך להפריד בין אימון לוולידציה?

בכל Epoch הרשת עוברת על דוגמאות האימון, מחשבת תחזיות, מודדת אובדן ומעדכנת משקולות. אחרי הסיום בודקים את המודל על קבוצת Validation ללא חישוב נגזרות. אם האובדן על Train ממשיך לרדת, אבל האובדן על Validation עולה, זו אינדיקציה אפשרית להתאמת יתר. אין להסיק מהתנהגות של Epoch יחיד מסקנה גורפת.

בדוגמת הקוד אנחנו בודקים את שתי הקבוצות אחרי כל Epoch. בספר נריץ מעט Epochs, כדי לשמור על זמן מעבדה סביר גם ללא GPU. בפרויקט גמר ניתן לשלב Batch קטן, Early Stopping ושמירת הגרסה הטובה ביותר לפי Validation. רצוי לא לבחור את המודל לפי Test, גם אם הדבר מפתה אחרי ניסוי ראשון.



איור: עקומות סכמטיות — אינן תוצאות אימון אמיתיות.

```
from torch.utils.data import TensorDataset, DataLoader

loader = DataLoader(TensorDataset(train_x, train_y),
                    batch_size=64, shuffle=True)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.003)

for epoch in range(5):
    model.train()
    running = 0.0
    for xb, yb in loader:
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        running += loss.item() * len(yb)

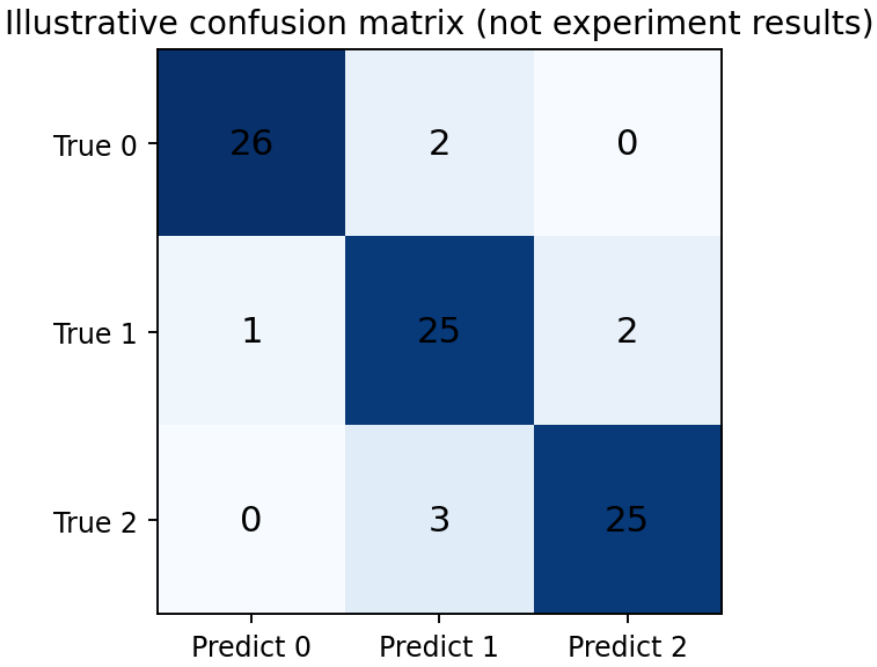
    model.eval()
    with torch.no_grad():
        val_logits = model(val_x)
        val_loss = criterion(val_logits, val_y).item()
        val_acc = (val_logits.argmax(1) == val_y).float().mean().item()
    print(epoch + 1, round(running / len(train_y), 4),
          round(val_loss, 4), round(val_acc, 3))
```

פלט המדדים תלוי באתחול, בסביבת הביצוע ובבחירות האימון; אין להבטיח תוצאה מסוימת. בדוגמת הפרק אין שימוש ב־*Test* לצורך בחירת *Epochs*.

16. כיצד נבדוק ביצועי CNN בצורה מדעית?

Accuracy מודד את שיעור התחזיות הנכונות, אבל אינו אומר אילו ספרות התבלבלו זו בזו. לכן נציג גם מטריצת בלבול, ונסתכל על כמה דוגמאות שסווגו בטעות. אם המודל מתקשה בספרות 3 ו־8, למשל, כדאי לשאול האם מדובר בכתב יד דומה, בעובי קו או בייצוג לא מספיק טוב.

מטריצת הבלבול שבתמונה הבאה היא דוגמה המחשה בלבד, ולא תוצאה שמדדנו במודל שלנו. במעבדה התלמיד יחשב מטריצה אמיתית מהתחזיות שלו, לאחר שהחליט על המודל. חשוב גם לחשוב על מדדי Macro F1 כאשר יש חוסר איזון משמעותי בין מחלקות.



איור: מטריצת בלבול מדגימה כיצד מנתחים את סוגי הטעויות.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Inspect errors on validation during model development.
model.eval()
with torch.no_grad():
    val_pred = model(val_x).argmax(1).cpu().numpy()
print('CNN validation accuracy:', accuracy_score(y_val, val_pred))
print(confusion_matrix(y_val, val_pred))
print(classification_report(y_val, val_pred, zero_division=0))
```

17. השוואה הוגנת למודל בסיס — האם CNN תמיד מנצחת?

לא. על מאגרים קטנים מאוד, מודל KNN עשוי להיות תחרותי ואף להצליח יותר מרשת CNN מסוימת. CNN דורשת בחירות ארכיטקטורה, אימון ומשאבי חישוב, ואילו KNN מבוסס על דוגמאות שמורות ומרחקים. השוואה מקצועית אינה מתחילה במסקנה אלא בניסוי הוגן: אותן קבוצות נתונים, אותן דרכי מדידה, ושימוש ב־Validation לצורך בחירת פרמטרים.

בדוגמת הספר שלנו נשווה תחזיות KNN לרשת CNN רק אחרי שהרשת עברה אימון. נשווה ביצועים, זמן הרצה, מספר פרמטרים וקשיים בניתוח טעויות. נדגיש שאין למדוד את ביצועי המודלים על Train ולדווח עליהם כיכולת הכללה.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train.reshape(len(X_train), -1), y_train)
knn_val_pred = knn.predict(X_val.reshape(len(X_val), -1))
knn_val_accuracy = accuracy_score(y_val, knn_val_pred)
cnn_val_accuracy = accuracy_score(y_val, val_pred)
print('KNN validation:', round(knn_val_accuracy, 3))
print('CNN validation:', round(cnn_val_accuracy, 3))

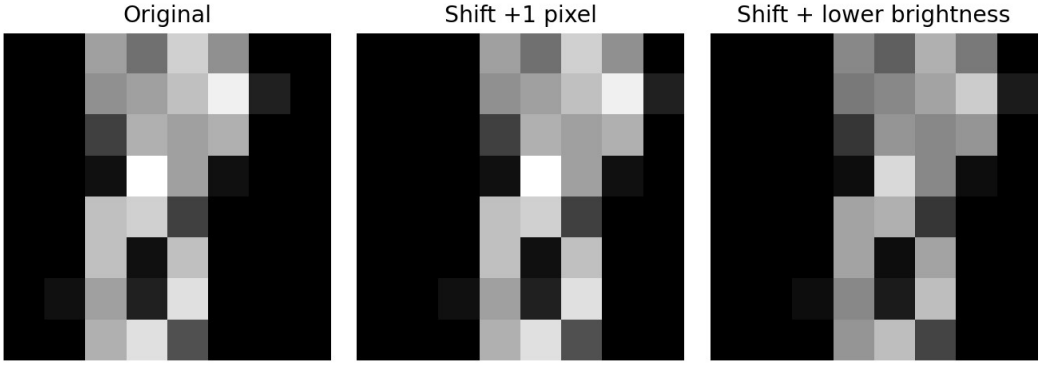
# Make the decision before looking at Test.
if knn_val_accuracy >= cnn_val_accuracy:
    selected = 'KNN'
    final_pred = knn.predict(X_test.reshape(len(X_test), -1))
else:
    selected = 'CNN'
    model.eval()
    with torch.no_grad():
        test_x = torch.tensor(X_test[:, None, :, :])
        final_pred = model(test_x).argmax(1).cpu().numpy()

print('Selected by validation:', selected)
print('Final test accuracy:', accuracy_score(y_test, final_pred))
print(confusion_matrix(y_test, final_pred))
```

18. Data Augmentation — למה להציג גרסאות נוספות של תמונה?

בפרויקט לזיהוי ספרות, צילום של אותה ספרה יכול להיות מעט מוזז או בעל בהירות שונה. אפשר להוסיף בזמן האימון שינויים ששומרים על התווית, כגון הזזה קטנה. זו שיטת Augmentation: יצירת דוגמאות אימון חדשות מתוך דוגמאות קיימות.

אסור להפעיל Augmentation על Test כדי להגדיל את מספר הדוגמאות המדווחות, ואסור לבחור שינויים שמשנים את משמעות הספרה. לדוגמה, סיבוב חד או שיקוף עלול ליצור ספרה אחרת או להפוך אותה ללא קריאה. על תלמיד בפרויקט להסביר מדוע הטרנספורמציות שבחר תואמות את המציאות הצפויה.



איור: המחשה של שינויי קלט אפשריים — לא כל שינוי מתאים לכל מאגר.

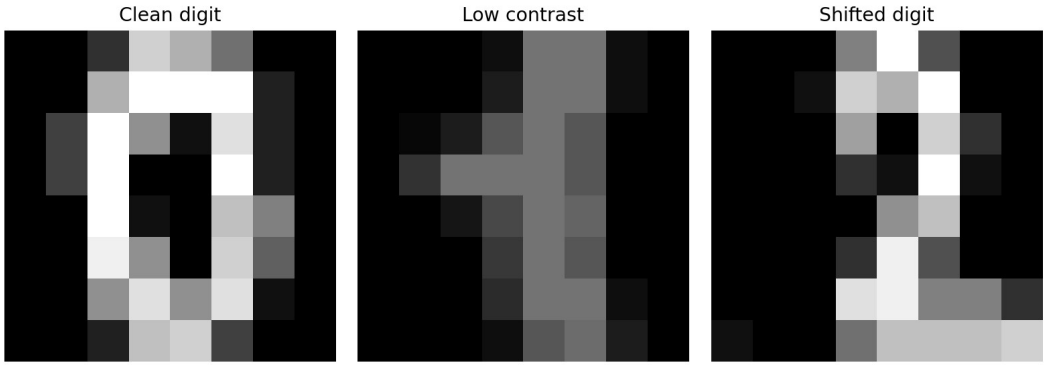
19. טעויות נפוצות שחייבים לזהות

טעות	מדוע זו טעות	כיצד מתקנים
שיטוח התמונה לפני Conv2d	נעלם המבנה המרחבי שהשכבה מנצלת	להשתמש ב־(N,C,H,W)
הוספת Softmax לפני CrossEntropyLoss	CrossEntropyLoss מקבלת Logits	להעביר ערכי Logits ישירים
שימוש ב־Test לבחירת Epoch	מידע מהבדיקה משמש לכיוון המודל	לבחור באמצעות Validation
בלבול בין מספר ערוצים לגודל התמונה	מספר מסננים קובע ערוצי פלט	לחשב בנפרד C_out ו־H_out,W_out
הנחה ש־CNN תמיד עדיפה על KNN	הצלחה תלויה בנתונים ובאימון	לבצע השוואה הוגנת
Augmentation שמשנה תווית	המודל לומד דוגמאות סותרות	לבחון כל טרנספורמציה בהקשר
שכחת model.eval() בזמן בדיקה	התנהגות של Dropout/BatchNorm יכולה להשתנות	להגדיר מצב בדיקה

20. חקר מקרה — מדוע המודל נכשל על תמונות שלא ראה?

נניח שרשת זיהתה תמונות ספרות מסריקות ברורות, אך במבחן על תמונות מטושטשות מאפליקציה חדשה הביצועים ירדו. לפני שמוסיפים שכבות יש לבדוק האם השוני נובע מתאורה, מרזולוציה, משיעור הספרות במאגר, מתהליך Crop, או מתוויות לא מדויקות. זהו שינוי התפלגות (Distribution Shift).

פתרון אפשרי אינו רק ״לאמן יותר״. ייתכן שצריך להגדיר מחדש תהליך עיבוד, להרחיב את נתוני האימון, לבצע הערכה לפי קבוצות או לשנות את הבעיה. בפרויקט גמר נדרשת יכולת לזהות את הסיבה האפשרית לכישלון ולהציע ניסוי מבוקר שיאמת אותה.



איור: שינויים בבהירות ובהזזה ממחישים מדוע יש לבדוק עמידות לשינויי קלט.

21. תרגול מדורג — הבנה, חישוב, ניתוח ונימוק

רמה א — הבנת מושגים

1. הסבירו בשפה שלכם מה ההבדל בין Kernel לבין Feature Map. מה מהם נלמד באימון?

2. הגדירו מהו Stride. הסבירו כיצד שינוי מ־1 ל־2 עשוי להשפיע על מספר תאי הפלט.

3. מהו ההבדל בין Max Pooling לבין Convolution מבחינת קיומן של משקולות נלמדות?

4. היכן מופיעה פונקציית ReLU ברשת, ומה קורה לפלט שלילי?

רמה ב — חישוב ידני

5. עבור מטריצה 5×5 ומסנן 3×3, Stride=1, Padding=0: חשבו את ממדי הפלט והסבירו את המספר.

6. עבור קלט 16×16×3 ו־12 מסננים בגודל 3×3, Padding=1: חשבו את מספר המשקולות ואת ממדי הפלט.

7. חשבו את ארבעת ערכי Max Pooling למטריצה 4×4 שבראש סעיף 10. השוו ל־Average Pooling.

8. לפלט Logits=[0,2,1] ולתווית אמת 1: בטאו את ה־Softmax של התווית ואת ה־Cross-Entropy. אפשר להיעזר במחשבון.

רמה ג — ניתוח מקצועי והכנה להגנה בעל־פה

9. תלמיד דיווח על Accuracy 99% בנתוני אימון ו־76% ב־Test. הציעו לפחות שלוש בדיקות שצריך לבצע לפני שינוי הארכיטקטורה.

10. השוו בין CNN ל־KNN בזיהוי ספרות: אילו יתרונות מעשיים יש לכל אחת, ומה אי אפשר להסיק ללא ניסוי?

11. מפתח הוסיף סיבוב של 180° לכל תמונת אימון. האם זה תמיד Augmentation תקין לספרות? נמקו.

12. בחרו בעיית תמונה לפרויקט גמר. הגדירו מאגר, תוויות, Baseline, חלוקת נתונים, שלוש בדיקות סיכון ושני מדדי הערכה.

22. תכנית הוראה מוצעת ל־50 שעות

זהו תכנון כיתתי מוצע של 10 יחידות בנות חמש שעות לימוד, לא חלוקה רשמית של מפגשי ההשתלמות.

יחידה	שעות	תוכן ותוצר
1	5	תמונות, טנזורים, קלט וידע קודם; מפת מושגים
2	5	Kernel וקורלציה צולבת; מפת מאפיינים בחישוב ידני
3	5	Padding, Stride, ערוצים וחישובי ממדים; בוחן קצר
4	5	Pooling, ReLU ואדריכלות; מעבדת NumPy
5	5	ארכיטקטורת CNN וספירת פרמטרים; פירוק שכבות
6	5	Forward, Loss, Backprop; ניסוי Logits
7	5	מימוש PyTorch, DataLoader ואימון מבוקר

יחידה	שעות	תוכן ותוצר
8	5	Validation, השוואה ל־KNN, מטריצת בלבול
9	5	Augmentation, ניתוח שגיאות ועמידות
10	5	מיני־פרויקט CNN, תיעוד והצגה בעל־פה
סה״כ	50	פרויקט למידה ותיק ניסויים

23. נספח לפרויקט הגמר — דף ניסוי CNN

מומלץ להדפיס או להעתיק את כרטיס הניסוי הבא לכל שינוי מהותי ברשת. מטרתו למנוע מצב שבו תלמיד משנה כמה רכיבים בבת אחת ואינו יודע מה גרם לשיפור. ניסוי איכותי מגדיר השערה, שומר את חלוקת הנתונים, מתעד את השינוי ומפריד בין תוצאות Validation לבין Test.

שדה	מה התלמיד מתעד
שאלת חקר	איזו בעיה אמיתית רוצים לפתור?
גרסת מאגר	מקור, גודל, רישוי, איכות ואופן חלוקה
ארכיטקטורה	שכבות, מסננים, Pooling ומספר פרמטרים
Hyperparameters	Learning rate, Epochs, Batch size, Seed
Baseline	מודל פשוט להשוואה
מדדי Validation	Accuracy, F1, Loss וניתוח טעויות
החלטה מקצועית	מה השתנה, מה נלמד ומה מגבלה
בדיקת Test סופית	תוצאה לאחר סיום כל בחירות המודל

24. הקשר לשלוש תכניות הלימוד

מסלול	מה התלמיד כבר יודע / לומד	הקשר לחומר המקושר
יחידה 3 — כיתה י׳	ייצוג נתונים, וקטורים, גרפים, KNN והערכת מודלים	תשתית קוד ו־EDA; CNN אינה דרישת חובה מזוהה ביחידה 3
יחידה 5 — נושאי ליבה	וקטורים, תקנון, מדדים ו־Validation	תשתית מתמטית ומושגית; CNN אינה מוכרזת כאן כחומר חובה ביחידה 5
התמחות למידה עמוקה	פרק 6 עיבוד תמונה; פרק 11 מתמטיקה; פרק 12 DNN; פרק 13 CNN	יישום בתמונות, פיתוח מודל והכנה לפרויקט גמר

25. מקורות מקצועיים והבחנה בין חובה להעשרה

מפת תכנית ההתמחות במאגר GitHub מגדירה את פרק 13 — CNN ואת היקפו (50 שעות). ההסברים מתבססים על רעיונות מקצועיים המופיעים ב־Dive into Deep Learning (פרקי CNN), Understanding

Deep Learning מאת Simon Prince (פרק 10), Hands-On Machine Learning מאת Aurélien Géron, ועל תיעוד PyTorch. הדוגמאות, המשימות והאיורים בספר מקוריים, ולא הועתקו מספרים מסחריים.

נושאים חיוניים ללמידה כאן: ייצוג טנזורים, מסננים, Convolution, Stride/Padding, Pooling, ארכיטקטורה, אימון בסיסי והערכת מודל. כהעמקה מוצעים חישובי Cross-Entropy ידניים, מורכבות פרמטרים, Augmentation וניתוח Distribution Shift. רק נוסח התכנית הרשמי המעודכן מוסמך לקבוע אילו פרטים הם חובה בבחינה או בפרויקט.

https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/13-Convolutional-Neural-Networks

https://en.d2l.ai/chapter_convolutional-neural-networks/conv-layer.html

https://en.d2l.ai/chapter_convolutional-neural-networks/padding-and-strides.html

https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

https://mitpress.mit.edu/9780262377102/understanding-deep-learning/